République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research

École Nationale Supérieure d'Informatique (ESI) — Algiers
Engineering Cycle — Computer Science Specialization — IoT / Robotics Integrative Project

EsiBot

Autonomous Differential-Drive Mobile Robot

Technical Report
Design, Integration and Experimental Evaluation
of an Autonomous Mobile Robot under ROS 2 Humble

Module: Internet of Things — Mobile Robotics

Academic Year: 2025–2026

Team Lead:
BOUMISSA Mohamed Chakib

Members:
ALLOUCHE Reda
BENDAOUD Anes
CHABANE Ouail
IDJA Rayane
MOSTEFAI Mounir Sofiane
TATI Youcef

Abstract

EsiBot is a differential-drive autonomous mobile robot designed and built by the team as part of the IoT integrative project at the École Nationale Supérieure d'Informatique (ESI Algiers). The initial specification was inspired by the TurtleBot3 Burger (ROBOTIS): an ESP32-CAM was meant to directly drive the motors, the servo motor and a rotating ultrasonic radar (HC-SR04 + SG90), while the Raspberry Pi 4 hosted the ROS 2 software stack (Simultaneous Localization and Mapping (SLAM), Nav2, web dashboard).

Over the course of development, the hardware architecture actually implemented evolved significantly from this specification. Control of the two DC motors, encoder readout and velocity control (Proportional-Integral-Derivative (PID)) are now handled *directly* by the General-Purpose Input/Output (GPIO) pins of the Raspberry Pi 4, assisted by an MPU6050 inertial measurement unit fused through an extended Kalman filter (`robot_localization`). The ESP32-CAM is retained solely as a Wireless Fidelity (IEEE 802.11) (WiFi) streaming camera (Motion JPEG (MJPEG)), no longer participating in actuator control. The rotating ultrasonic radar remains available as a slow fallback path, but current mapping relies on a Kinect (v1, Xbox 360 era) depth sensor, driven by `kinect_ros2` and converted to a laser scan via `depthimage_to_laserscan`. The `working_build` branch also adds the `depth_cleanup` library to prepare a cleaned depth stream, and references `m-explore-ros2` for autonomous exploration.

This report faithfully documents this as-built architecture: mechanical design (differential chassis described by the Unified Robot Description Format (URDF) model), electronic wiring inferred from code parameters, low-level control and motor control loops, the perception pipeline (Inertial Measurement Unit (IMU), Kinect), SLAM mapping, autonomous ROS 2 Nav2 navigation, a telemetry web dashboard, and Xbox controller teleoperation. Measured results (or results derived analytically from the parameters declared in the code, where field measurements were still in progress) are presented, followed by a critical analysis of the discrepancies between the original design and the actual implementation, the identified limitations, and an outlook for future improvements.

Keywords: ROS 2 Humble, differential-drive robot, SLAM, slam_toolbox, Nav2, Kinect, Raspberry Pi 4, ESP32-CAM, MPU6050, extended Kalman filter, autonomous navigation, IoT robotics, TurtleBot3.

Contents

Abstract . . . . . **i**
1 Introduction **1**
1.1 Project Objectives 1
1.2 Report Organization 1
2 Overall Architecture **2**
2.1 Detailed Final Architecture 2
2.2 ROS Node Graoh 2
3 Mechanical Design & Wiring **3**
3.1 Mechanical Structure 3
3.2 Electrical Scheme 3
4 Low-Level Control & Odometry **4**
4.1 PID Speed Control 4
4.2 Sensor Fusion and Localization 4
5 SLAM, Sensing & Navigation **5**
5.1 Kinect Depth into Laser Scans 5
5.2 Mapping Process (slam_toolbox) 5
5.3 Navigation and Nav2 5
6 User Interface & Deployment **6**
6.1 Web Dashboard 6
6.2 System Launchers 6
7 Testing & Results **7**
7.1 Odometry Benchmarks 7
7.2 Navigation Constraints 7
7.3 Demonstration Scenario 7
8 Limitations **8**
8.1 Hardware and Power Management 8
8.2 Kinect Sight Bounds 8
8.3 Ultrasonic Latency 8
9 Conclusion & Action Plan **9**
9.1 Prioritized Action Plan 9
A Source Code & Links **10**
A.1 Repository Information 10
A.2 Hardware Schematics 10

CHAPTER 1

Introduction

1.1 Context

The **EsiBot** project is part of an IoT / Mobile Robotics integrative module at the École Nationale Supérieure d'Informatique (ESI Algiers). The objective of this project is to design, build and program an autonomous mobile robot capable of mapping an unknown environment and navigating within it to defined goal points. This project is inspired by the architecture of the **TurtleBot3 Burger**, replacing its proprietary hardware components with accessible, low-cost solutions: an ESP32-CAM Microcontroller Unit (MCU) microcontroller, a Raspberry Pi 4, a Kinect depth sensor, and 3D printing for the chassis.

1.2 Report Objectives

This document details the hardware, software and mechanical architecture of the system. It highlights the final design choices that led to the robot's implementation, in particular the integration of motor control on the Raspberry Pi 4 and the use of the Kinect sensor for mapping.

The report covers:

- The complete architecture (mechanical, electronic, and software components);
- Sensor interfacing and communication flows under ROS 2;
- Configuration parameters and sensor fusion;
- Performance evaluation and the system's limitations.

1.3 Report Organization

This report is structured as follows: Chapter 2 details the overall architecture. Chapter 3 covers the mechanical and electronic design. Chapter 4 documents low-level control and data fusion. The mapping, navigation and exploration approach is covered in Chapter 5. Chapter 6 presents the user interface and deployment procedures. Finally, the integration results, limitations analysis, and project outlook are addressed in the last three chapters (7, 8, 9).

CHAPTER 2

Overall System Architecture

2.1 Architecture Overview

The robot's hardware architecture is organized into three processing layers and a dedicated power layer. Figure 2.1 illustrates the overall architecture diagram.

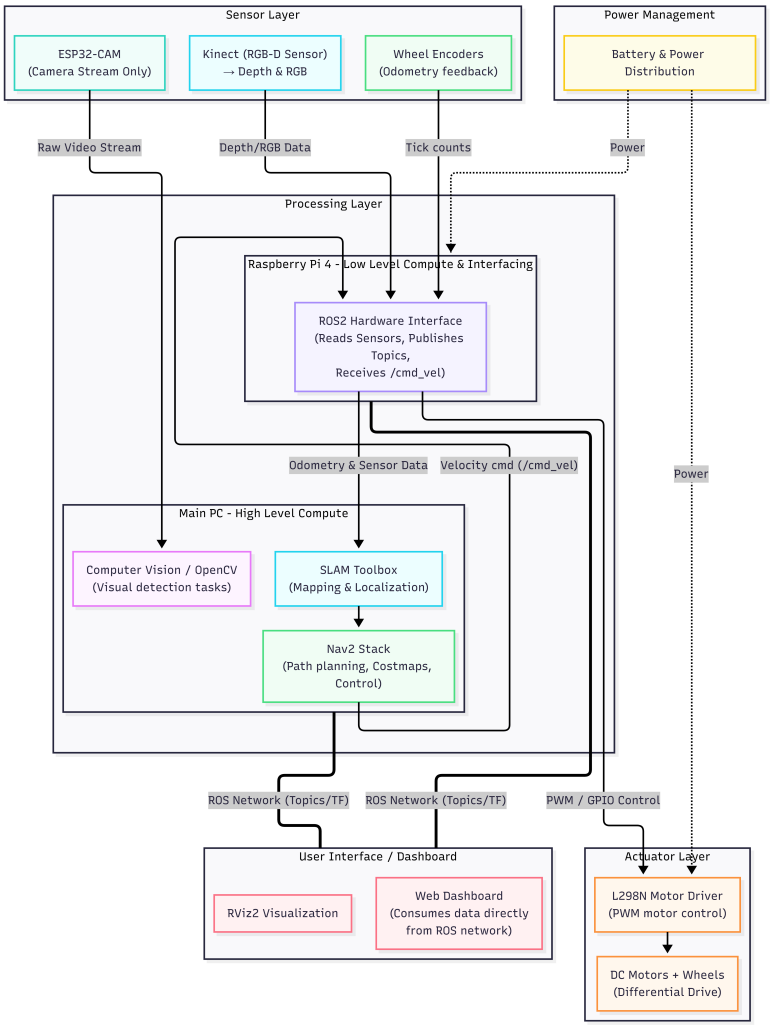


Figure 2.1. Overall **EsiBot** architecture. The primary mapping sensor is the Kinect (RGB-D) and the motors are driven by the Raspberry Pi 4.

The major components are structured as follows:

1. **Camera Server (ESP32-CAM):** Exposes the MJPEG video stream *via* HTTP over the local WiFi network.
2. **Low-Level Control (Raspberry Pi 4):** Directly drives the H-bridge (L298N) and reads motor odometry via the `motor_bridge_node` node.
3. **Primary Mapping Sensor (Kinect):** Provides the depth image, which can be cleaned by `depth_cleanup` and then converted into a laser scan via `depthimage_to_laserscan` to feed the SLAM system.
4. **Inertial Measurement Unit (IMU MPU6050):** Coupled with odometry via an Extended Kalman Filter (EKF) filter to stabilize the robot's heading and ensure reliable positioning for Nav2.
5. **Autonomous Exploration:** The `working_build` repository references `m-explore-ros2`, intended to sit on top of `/map` and Nav2 to automatically generate goals toward unknown frontiers.

2.2 Three-Layer Breakdown

Layer 1 — Sensor Layer. Three independent data sources feed the processing layer: the ESP32-CAM (raw MJPEG video stream), the Kinect (depth image and RGB color image), and the wheel encoders (pulse counting for odometry). Each source uses a distinct physical interface: WiFi for the camera, USB for the Kinect, interrupt-driven GPIO for the encoders.

Layer 2 — Low-Level Processing (Raspberry Pi 4). The Raspberry Pi 4 hosts the ROS 2 hardware interface: it reads the onboard sensors (encoders, IMU), runs the PID velocity control loop for both motors, publishes odometry and telemetry topics, and receives velocity commands on `/cmd_vel`.

Layer 3 — High-Level Processing (Operator PC). The official diagram places computer vision (OpenCV/`depth_cleanup`), slam_toolbox, `m-explore-ros2` and the Nav2 stack on a separate main PC, communicating with the Raspberry Pi 4 over the ROS 2 network (DDS, topics/Transform Tree (TF)). This logical separation is made possible by the distributed nature of ROS 2: the launch arguments (`start_base`, `start_kinect`, `start_slam`, `start_rviz`) of the `mapping.launch.py` file can be independently enabled, which allows running the heavy nodes (SLAM, Nav2) on a machine more powerful than the Raspberry Pi 4, while leaving the latter dedicated to real-time hardware interfacing.

Cross-Cutting Layer — Power. A battery and a regulated power distribution independently supply the actuator layer (motors via L298N) and the compute layer (Raspberry Pi 4, ESP32-CAM), as shown in Figure 2.1.

2.3 ROS 2 Node and Topic Graph

Figure 2.2 shows the simplified graph of nodes and topics implemented in the system. Processing and data fusion follow ROS 2 standards, linking the hardware interfaces to high-level planning.

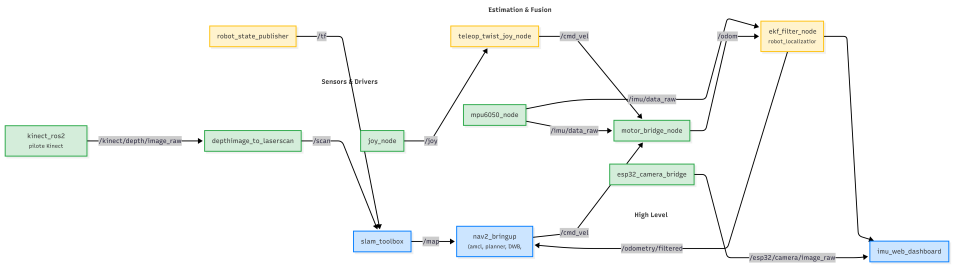


Figure 2.2. Simplified ROS 2 node and topic graph. Colors: green = drivers, blue-gray = state estimation, blue = planning/mapping.

2.4 Summary of Main Topics

Table 2.1 summarizes the ROS 2 communication topics. The exhaustive list is provided in Appendix A.

Table 2.1. Main ROS 2 topics.

Topic	Type	Published by	Subscribed by
`/cmd_vel`	geometry_msgs/Twist	teleop_twist_joy, Nav2	motor_bridge_node
`/odom`	nav_msgs/Odometry	motor_bridge_node	ekf_filter_node
`/odometry/filtered`	nav_msgs/Odometry	ekf_filter_node	Nav2, imu_web_dashboard
`/imu/data_raw`	sensor_msgs/Imu	mpu6050_node	motor_bridge_node, ekf_filter_node, dashboard
`/scan`	sensor_msgs/LaserScan	depthimage_to_laserscan (Kinect)	slam_toolbox, Nav2 costmaps
`/map`	nav_msgs/OccupancyGrid	slam_toolbox / map_server	Nav2, RViz2, m-explore
`/tf`, `/tf_static`	tf2_msgs/TFMessage	robot_state_publisher, ekf, slam_toolbox	Nav2, RViz2
`/joy`	sensor_msgs/Joy	joy_node	teleop_twist_joy_node, dashboard
`/esp32/camera/image_raw`	sensor_msgs/Image	esp32_camera_bridge	(optional consumer)

CHAPTER 3

Mechanical and Electronic Design

3.1 Differential Architecture

EsiBot adopts a *differential drive* architecture: two independent driven wheels plus a caster wheel ensuring static stability, with steering resulting solely from the speed difference between the two wheels. The chassis Computer-Aided Design (CAD)/STL files form a separate deliverable (Deliverable 1) not included in the analyzed code; this chapter therefore relies on the URDF model (`esibot_description`), the source of truth used by ROS 2 for transforms and costmaps, supplemented by the electronic constants found in the control code.

3.2 Geometric Model

Table 3.1. Geometric and inertial parameters declared in the URDF model.

Element	Value	Note
Chassis (`base_link`)	0.21 × 0.14 × 0.03 m	mass 2.0 kg
Wheel radius (`wheel_radius`)	0.025 m	width 0.027 m, mass 0.2 kg/wheel
Wheelbase (`wheel_y` ×2)	0.14 m	lateral distance between wheels
Camera module (`camera_link`)	0.073 × 0.278 × 0.073 m	mass 0.56 kg, $x = -0.105$ m (rear)

The model declares the camera optical frames (`camera_depth_frame`, `camera_rgb_optical_frame`, $-\pi/2$ rotation); a `static_transform_publisher` links `camera_depth_frame` to the Kinect driver's native frame `kinect_depth` via an identity transform.

Limitation

A revision of the geometric parameters (in particular the wheel radius and the wheelbase) is needed to guarantee full consistency between the URDF model and the low-level control code, in order to optimize odometry accuracy.

3.3 Differential Kinematics and Odometry

The `/cmd_vel` → wheel speed conversion, implemented in `motor_bridge_node.py` (`twist_to_wheel_commands`) and replicated in the dashboard (`imu_web_dashboard.py`), follows the classic model:

$$v_l = v - \omega \cdot \frac{L}{2}, \quad v_r = v + \omega \cdot \frac{L}{2} \tag{3.1}$$

and conversely for odometry reconstruction: $v = (v_l + v_r)/2$, $\omega = (v_r - v_l)/L$ (equation 3.2), where L is the wheelbase.

$$v = \frac{v_l + v_r}{2}, \qquad \omega = \frac{v_r - v_l}{L} \tag{3.2}$$

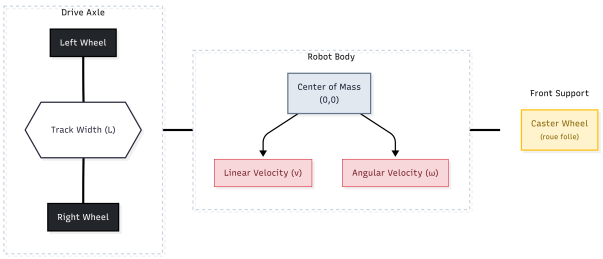


Figure 3.1. Kinematic model of the **EsiBot** differential robot (top view).

Each wheel is fitted with an encoder counted via GPIO interrupt, with 2 ms software debouncing. The `encoder_cpr` parameter (12 by default) is doubled by counting on both edges (`encoder_count_both_edges=True`), giving **24 pulses/revolution**. The measured rotation speed is computed at each control cycle (`control_hz`, 10 Hz by default) using:

$$\text{RPM}_{\text{measured}} = \frac{N_{\text{pulses}}}{\text{CPR}_{\text{effective}}} \times \frac{60}{\Delta t} \tag{3.3}$$

i.e. $360°/24 = 15°$ per pulse, and, with the calibrator's 0.033 m radius, ≈**8.6 mm**/pulse ($2\pi \times 0.033/24$).

3.4 Electronic Wiring

The specification (Deliverable 2) requires a Fritzing/KiCad schematic, which is absent from the provided archive; Table 3.2 was therefore **reconstructed by reverse engineering** from the GPIO constants in the Python code. Unlike the specification, which planned for the ESP32-CAM to drive the H-bridge, the servo and the HC-SR04, the actual implementation concentrates all real-time electronics on the GPIO pins of the Raspberry Pi 4.

Table 3.2. Raspberry Pi 4 GPIO pins (BCM), reconstructed from the source code.

Function	GPIO	Source / node
ENA / IN1 / IN2 (left motor)	12 / 5 / 6	`motor_bridge_node`, `motor_calibrator`
Left encoder	26	`motor_bridge_node` (`pin_left_encoder`)
ENB / IN3 / IN4 (right motor)	13 / 21 / 20	`motor_bridge_node`, `motor_calibrator`
Right encoder	27	`motor_bridge_node` (`pin_right_encoder`)
I2C SDA/SCL (bus 1)	2 / 3	`mpu6050_node` (MPU6050 @ 0x68)
HC-SR04 SIG (trig/echo)	23	`ultrasonic_scan_node` (deprecated)
SG90 radar servo	18	`ultrasonic_scan_node` (deprecated)

In the `working_build` branch, the calibrator's software wiring and that of the production node are consistent: both use BCM pins 26 and 27 for the left and right encoders. The PWM→RPM calibration can therefore directly feed `pwm_mapper.py`, provided the wheel radius and wheelbase used by the odometry are experimentally verified (see Chapter 8).

Motor driving uses a dual **L298N** H-bridge with software PWM (`RPi.GPIO`, 500 Hz), with duty cycle bounded to $[15\,\%, 100\,\%]$ (`min/max_duty_cycle`) to compensate for mechanical dead zone. The MPU6050 (I2C bus 1, address `0x68`) is configured at startup (wake-up, sample-rate divider `SMPLRT_DIV=0x07`, low-pass filter `DLPF_CFG=0x03`, $\pm 2g/\pm 250°$/s) and publishes at **50 Hz**. The Kinect (v1, Xbox 360 era), the primary mapping sensor, is connected via **USB** (not GPIO) and driven by the community package `kinect_ros2` (`libfreenect`).

3.5 Camera Server (ESP32-CAM)

The ESP32-CAM module (`AI_THINKER`) is used exclusively as a WiFi MJPEG video server: the `CameraServerArduinoIDE.ino` firmware initializes the OV2640 sensor (`UXGA` if PSRAM is present, otherwise `SVGA`) and exposes the stream on `/stream`. No GPIO pin of the ESP32-CAM drives an actuator — most of its pins being reserved for the parallel camera interface — a hardware constraint likely behind the move of motor control to the Raspberry Pi 4. The WiFi credentials (SSID/password) are **hardcoded** in the firmware, with no captive portal or provisioning (Chapter 8).

3.6 Power Supply

The architecture diagram (Figure 2.1) identifies a "Power Management" block separately supplying the actuator layer (L298N) and the compute layer (Raspberry Pi 4, ESP32-CAM). No battery supervision node (`/battery_state`, planned in the specification) was found in the code: battery telemetry is, at this stage, not exposed by the onboard software.

CHAPTER 4

Low-Level Control and Sensor Fusion

4.1 Motor Control (`motor_bridge_node`)

This node (package `esibot_hardware`) is the core of the robot's real-time control. On the Raspberry Pi 4, it plays the role originally assigned to the ESP32/`esibot_driver` pair in the specification: receiving `/cmd_vel`, wheel velocity control, encoder reading, and publishing raw odometry.

4.1.1 Control Loop

The node runs a periodic loop at `control_hz` (10 Hz by default) which, on each cycle:

1. reads the pulse counts accumulated since the last cycle for each wheel;
2. computes the measured rotation speed in rpm (equation 3.3);
3. applies a PID controller (one per wheel) to bring the measured speed closer to the setpoint derived from the differential kinematics (equation 3.1);
4. converts the controller output into a PWM duty cycle via `pwm_mapper.py`;
5. applies an optional heading correction based on the IMU (yaw control);
6. publishes the integrated odometry on `/odom` and the counter status on the diagnostic topics (`/encoder_ticks\protect\penalty\z@/left`, `/encoder_ticks\protect\penalty\z@/right`).

4.1.2 Per-Wheel PID Controller

Each wheel has its own independent PID controller, with default gains K_p, K_i, K_d configurable via the ROS 2 parameter server (and adjustable during operation without recompilation). The controller output is saturated, and an anti-*windup* mechanism limits accumulation of the integral term when the command is saturated, avoiding overshoot during abrupt setpoint changes.

Safety Loop

The node implements a *watchdog* timer: if no `/cmd_vel` message is received for a duration `cmd_vel_timeout` (typically a few hundred milliseconds), both motors are automatically cut off. This mechanism protects the robot in case of network communication loss with the teleoperation node or Nav2.

4.2 Wheel Speed to PWM Conversion

The `pwm_mapper.py` module converts a desired linear wheel speed (in m/s) into a normalized PWM duty cycle. In `working_build`, this conversion is no longer a simple theoretical calibration: it loads the `~/.esibot/pwm_rpm_map.json` file, generated on the robot, and applies a separate affine regression for each wheel and direction of travel. The conversion combines:

- a mechanical dead zone (`min_duty_cycle`, 15 % by default and 22 % in some launches without calibration);
- a/b coefficients calibrated for `left/right` and `forward/reverse`, to compensate for efficiency differences between the two motors;
- a cap at `max_pwm`/`max_rpm` derived from calibration, which avoids commanding into a saturation or mechanical-noise region.

4.3 Calibration Tool (`motor_calibrator.py`)

A standalone calibration script (outside the ROS 2 graph, run from the command line) empirically characterizes the relationship between PWM duty cycle and actual rotation speed (in rpm) for each motor. It sweeps duty cycles from 15 % to 100 % in 5 % steps, measures two one-second samples per point, keeps the median, then stops the sweep as soon as an RPM drop indicates saturation or an unstable reading. The result is saved to `~/.esibot/pwm_rpm_map.json` along with the wheel radius used (0.033 m), the PWM frequency (500 Hz) and the forward/reverse coefficients for each wheel. This tool is essential for limiting motor mechanical asymmetry, but it alone is not sufficient to guarantee correct odometry if the actual wheel radius, wheelbase and SLAM parameters are not recalibrated together (Chapter 8).

4.4 Inertial Measurement Unit and Heading Stabilization

The `mpu6050_node` node (package `esibot_sensors`) starts with a stationary gyroscope bias calibration over 100 samples, then continuously publishes (50 Hz):

- `/imu/data_raw` (`sensor_msgs/Imu`) — raw linear accelerations (a_x, a_y, a_z) and angular velocities $(\omega_x, \omega_y, \omega_z)$;

- `/imu/temperature` (`sensor_msgs/Temperature`) — internal sensor temperature, useful for thermal diagnostics.

`motor_bridge_node` subscribes to `/imu/data_raw` to apply closed-loop yaw correction when the robot is moving forward and no deliberate rotation is commanded. The controller compensates for residual yaw rate measured by the IMU, with a dead band of 0.02 rad/s and a maximum correction of 40 RPM per wheel.

4.5 Data Fusion (EKF Filter)

Raw odometry (`/odom`, from encoders alone) and inertial data (`/imu/data_raw`) are fused by an EKF provided by the standard `robot_localization` package (`ekf_filter_node` node), configured in two-dimensional mode (`two_d_mode: true`). The `working_build` `ekf_odom_imu.yaml` file is deliberately conservative: it fuses the odometry velocities $(v_x, v_y, \dot{\theta})$ and only the yaw rate from the IMU. It injects neither absolute orientation nor linear acceleration, since the MPU6050 does not publish an estimated orientation and its raw acceleration contains gravity and residual bias. The `/odometry/filtered` output thus provides a more stable local odometry for SLAM/Nav2, while `slam_toolbox` or Adaptive Monte Carlo Localization (AMCL) retain responsibility for the global `map` $\rightarrow$ `odom` correction.

CHAPTER 5

Mapping and Autonomous Navigation

5.1 Kinect to SLAM Pipeline

The system relies on a **Kinect** depth sensor to generate the environment map. The extracted depth data is converted into a 2D laser scan. This approach provides the standard `sensor_msgs/LaserScan` message structure expected by the SLAM algorithm. The `mapping.launch.py` file launches and orchestrates this entire chain (Figure 5.1).

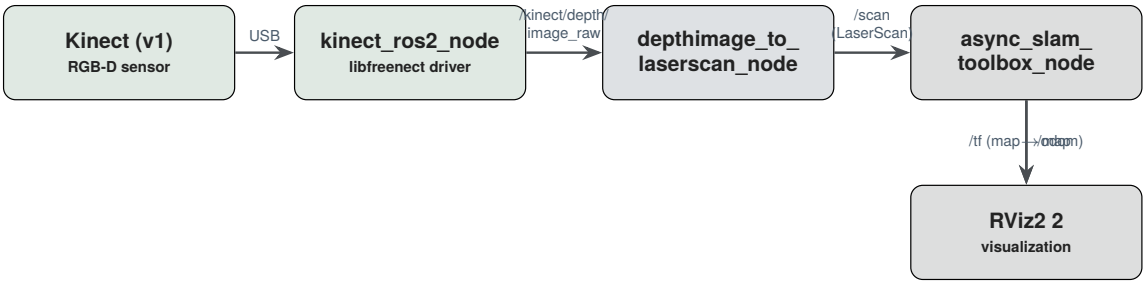


Figure 5.1. Mapping pipeline as orchestrated by `mapping.launch.py`.

The `kinect_ros2` driver (namespace `kinect`, based on `libfreenect`) exposes `/kinect/depth/image_raw` and `/kinect/depth/camera_info`. A `static_transform_publisher` links `camera_depth_frame` (URDF) to the native `kinect_depth` frame via an identity transform. The `depthimage_to_laserscan_node` node then projects a horizontal band of the depth image into a synthetic laser scan (Table 5.1).

Table 5.1. Parameters of `depthimage_to_laserscan_node` (`mapping.launch.py`).

Parameter	Value	Meaning
`scan_height`	10 px	band of the depth image averaged per scan
`scan_time`	0.033 s	nominal period (≈30 Hz)
`range_min` / `max`	0.35 / 4.0 m	usable Kinect range
`output_frame`	`camera_depth_frame`	TF frame of the `LaserScan`

Reduced Field of View

The Kinect covers $\approx 57^\circ$ horizontally compared with the 180–360 degrees planned for a rotating radar or a panoramic LiDAR: the lateral/rear zones remain blind at any given instant, requiring active rotation of the robot during mapping to compensate.

5.2 Depth Preprocessing with `depth_cleanup`

The `working_build` branch adds the `depth_cleanup` package, a C++/OpenCV library independent of ROS 2 and `libfreenect`. Its role is to clean up holes in the Kinect depth data while preserving a metric `CV_16UC1` image in millimeters, usable thereafter by a ROS 2 node that would publish a `sensor_msgs/Image` synchronized with `CameraInfo`. The geometry mode adopted is `applyKinectSmoothingPipeline()`:

- null pixels are treated as invalid;
- a 3×3 neighborhood fills small holes using a mode quantized into 10 mm bins;
- remaining holes are filled with `cv::INPAINT_TELEA` at 20 % resolution to limit CPU cost;
- the original valid pixels remain unchanged, which is checked by the package's unit tests.

The `SLAM_INTEGRATION.md` document explicitly sets the integration boundary: `slam_toolbox` does not consume the depth image directly, but the `LaserScan` produced by `depthimage_to_laserscan`. At the current state of the `mapping.launch.py` launch, the remapping still points to `/kinect/depth/image_raw`. The remaining step therefore consists of inserting a ROS 2 adapter that publishes the cleaned `16UC1` image and its `CameraInfo`, then remapping `depthimage_to_laserscan` to this cleaned topic.

5.3 Why the Kinect Replaces the Ultrasonic Radar

The ultrasonic path remains present in `ultrasonic_scan_node`, but it is not suited to fast autonomous driving. The `working_build` configuration sweeps only from -30° to $+30^\circ$ in 5° steps, takes five measurements per angle, waits 0.35 s for servo settling, and publishes roughly 0.07 scans per second. Such a sensor

gives a sequential scalar distance per direction, strongly dependent on the servo's instantaneous orientation: a chair, a table leg, a low obstacle or a sloped surface can be missed or mapped to the wrong location between two costmap updates.

The article by Kamarudin et al. on the analysis of the Kinect for 2D SLAM, referenced in Appendix A.3, notes that a Kinect sensor used for 2D SLAM remains limited by its field of view and range, but that its 3D depth data offers a richer basis for detecting objects of varying shapes and sizes. For **EsiBot**, this transforms the problem: instead of building a costmap from a slow ultrasonic beam, a coherent LaserScan slice is extracted from a complete depth scene updated at video rate. The ultrasonic radar therefore still has value as a diagnostic tool or a slow local safety check, but the Kinect is the consistent sensor feeding `slam_toolbox`, Nav2 and autonomous exploration.

5.4 `slam_toolbox` Configuration

`slam_toolbox_esibot_kinect.yaml` configures `async_slam_toolbox_node` in `mapping` mode (Ceres solver, `SPARSE_NORMAL_CHOLESKY`). Table 5.2 summarizes the key parameters.

Table 5.2. Key `slam_toolbox` parameters for **EsiBot**.

Parameter	Value	Role
`resolution`	0.05 m	cell size of the occupancy grid
`min/max_laser_range`	0.35 / 4.0 m	consistent with `depthimage_to_laserscan`
`minimum_time_interval`	0.5 s	minimum interval between processed scans
`minimum_travel_distance/heading`	0.15 m / 0.15 rad	anti-oversampling filter
`do_loop_closing`	`true`	loop closure enabled (radius 3.0 m)
`transform_publish_period`	0.02 s	`map`→`odom` publication (≈50 Hz)

`use_map_saver: true` enables on-demand saving (`map_saver_cli`) as a `map.pgm`/`map.yaml` pair, subsequently consumed by Nav2.

Limitation

The current map used by `esibot_navigation2` (resolution 0.05 m, origin $(-10, -10, 0)$) corresponds, upon visual inspection, to the default TurtleBot3/Nav2 sample hexagonal map, and not to an actual map captured by **EsiBot**'s Kinect. The SLAM → save → Nav2 reload chain therefore still needed field validation at the time of writing (Chapter 7).

The RViz2 configuration `esibot_slam_map.rviz` is the project's main visualization tool (map, `/scan` scan, URDF model, full TF tree), rather than an integrated mapping view in the web dashboard (Chapter 6).

5.5 Autonomous Navigation with Nav2

Autonomous navigation relies on the standard **Nav2** stack (`navigation2.launch.py`, package `esibot_navigation2`), loading the static map, the `esibot.yaml` file, and orchestrating localization, costmaps, planner, controller, behaviors, velocity smoother and collision monitor.

Localization uses AMCL (particle filter, 500–2000 particles, `likelihood_field` model, `DifferentialMotionModel`, initial pose not set automatically). The local (3×3 m window) and global costmaps share a resolution of **0.05 m/cell**, a robot radius of **0.2 m**, and an inflation radius of **0.35 m** (factor 5.0). The global planner `NavfnPlanner` (Dijkstra, 0.5 m tolerance) feeds the Dynamic Window-Based local planner (DWB) local controller, whose bounds are summarized in Table 5.3.

Table 5.3. DWB and AMCL kinematic bounds (excerpt from `esibot.yaml`).

Parameter	Value	Unit
Min. / max. linear velocity (DWB)	0.2 / 0.3	m/s
Max. angular velocity (DWB)	0.5	rad/s
Linear acceleration / deceleration	3.0 / -2.5	m/s^2
Goal position tolerance (`FollowPath`)	0.05	m
Max. velocity (velocity smoother)	0.5 / 2.5	m/s / rad/s

The DWB scoring criteria (`PathAlign` 32, `GoalDist` 24, `BaseObstacle` 0.02, `RotateToGoal` 32) clearly

favor adherence to the global path over speed, consistent with a small-footprint indoor robot. The behavior server exposes `Spin`, `BackUp`, `DriveOnHeading`, `Wait`, `AssistedTeleop`; a `collision_monitor` watches a stop disk (radius 0.1 m), a slowdown polygon and an approach polygon (2 s horizon) before final transmission to `/cmd_vel`.

As noted in Section 5.4, the default loaded map is a generic sample map; the Nav2 configuration itself is complete and consistent with the Kinect pipeline, but its experimental validation on a real map still needed to be carried out (Chapter 7).

5.6 Autonomous Exploration

The `working_build` branch also adds `src/m-explore-ros2` as a Git dependency. This module corresponds to the frontier-based exploration layer: it sits on top of the `/map` map, the TF tree and Nav2, selects frontiers between known and unknown areas, then sends successive goals to the navigator. No `esibot_*` launch file orchestrates it yet alongside the bringup, the Kinect and Nav2; its operational integration must therefore verify three conditions before demonstration: the submodule initialized in the workspace, stable `/map`/`/tf` topics, and Nav2 active with either a map or online SLAM.

CHAPTER 6

User Interface and Implementation

6.1 Xbox Controller Teleoperation

Teleoperation relies on a wireless (Xbox-compatible) controller, driven by `joy_node`/`teleop_twist_joy` (`joystick_drive.launch.py`), rather than the `teleop_twist_keyboard` suggested in the specification.

Table 6.1. Controller teleoperation configuration (`xbox_teleop.yaml`).

Parameter	Value	Note
`require_enable_button`	`false`	no ”dead-man” button required
Linear / angular axis	axis 1 (scale 0.25) / axis 3 (scale 2.4)	sticks → `/cmd_vel`
Dead zone / repeat rate	0.08 / 20 Hz	`joy_node`

The absence of a mandatory enable button means that any accidental stick deflection immediately translates into motion, with no button to hold down — a reduced safety margin compared to a classic ”dead-man” device (Chapter 8). `joystick_drive.launch.py` jointly starts `joy_node`, `teleop_twist_joy_node` and `motor_bridge_node` (max angular velocity raised to 3.0 rad/s, minimum duty cycle 22 %).

6.2 EKF Fusion — Configuration Detail

`ekf_odom_imu.yaml` (`ekf_fusion.launch.py`) configures `ekf_filter_node` at **30 Hz**, `two_d_mode`: from odometry (`/odom`), only the *velocities* $(v_x, v_y, \dot{\theta})$ are fused; from the IMU (`/imu/data_raw`), only $\dot{\theta}$ is kept. This configuration avoids integrating the MPU6050’s raw acceleration or an orientation that the node does not provide. The `world_frame` remains `odom`, leaving the `map`→`odom` correction to `slam_toolbox`/AMCL — a clean separation between local (EKF) and global (SLAM/AMCL) estimation.

6.3 Telemetry Web Dashboard

The `imu_web_dashboard` node (`esibot_dashboard`) exposes a standalone HTTP server (`http.server`, no external dependency), serving a single-page HTML/JS page, noticeably different from the dashboard envisioned in the specification (which called for a `rosbridge_suite` WebSocket gateway with a map, video and teleoperation buttons) — the actual implementation focuses on **telemetry**.

Table 6.2. Widgets of the `imu_web_dashboard`.

Widget	Content
Orientation	roll/pitch (accelerometer integration), <canvas> artificial horizon, zeroing (`POST /api/zero`)
Odometry	position/heading/velocities (`/odometry/filtered`), trajectory (1200 pts)
Controller & command	`/joy` axes/buttons and `/cmd_vel` → wheel speed conversion (equation 3.1)

The interface continuously polls `GET /api/imu` (consolidated JSON state, lock-protected for consistency between the ROS 2 thread and the HTTP thread). The dashboard **shows neither the SLAM map nor the video stream**, two features of Deliverable 5 (Chapter 8).

6.4 ESP32-CAM Video Stream

The `esp32_camera_bridge` bridge (`mjpeg_image_publisher`) connects as an HTTP client to the ESP32-CAM’s MJPEG stream, extracts each JPEG frame (`0xFFD8`/`0xFFD9` markers), decodes it with OpenCV and republishes it on `/esp32/camera/image_raw` via `cv_bridge`, with automatic reconnection (2 s). This stream is consumable but is not consumed by any dedicated vision node — no `vision_node` (line/obstacle detection, planned in the specification) was found in the delivered code.

6.5 Implementation and Operating Procedures

The repository provides several shell scripts (`scripts/`) that illustrate the low-level/high-level separation, unlike the T1 test in the specification, which assumes a single `esibot_full.launch.py`.

Table 6.3. Startup scripts available in `scripts/`.

Script	Role	Notes
`start_esibot.sh`	manual launch of the calibrator	minimal development script
`start_esibot_all.sh`	production startup (teleop, IMU, EKF, dashboard)	requires valid calibration; radar commented out in the script
`start_esibot_all_no_calib.sh`	variant without verification	starts the ultrasonic radar in addition to teleoperation
`start_esibot_all_no_calib_no_imu.sh`	bringup without mandatory calibration	IMU, EKF and dashboard enabled via environment variables
`start_esibot_keyboard_autonomous.sh`	keyboard/autonomy command mode	no joystick; expects an external `/cmd_vel` publisher
`read_pwm_pins.py`, `read_wheel_encoders.py`	GPIO diagnostics	direct reading of PWM signals and encoder ticks

No script starts the Kinect/SLAM pipeline

The shell scripts are limited to the ”low-level control + telemetry” layer (Chapter 4). The mapping pipeline (`mapping.launch.py`) and Nav2 (`navigation2.launch.py`) must be started *separately*, typically from the operator PC (high-level layer, Figure 2.1) — a direct reflection of the three-layer breakdown (Chapter 2).

Before any startup via `start_esibot_all.sh`, the script verifies the validity of a JSON calibration file (`~/.esibot/pwm_rpm_map.json`, generated by `motor_calibrator`, Section 4.3), with positive coefficients required for the 4 {left,right}×{forward,reverse} combinations — a safeguard that blocks startup in case of missing or invalid calibration.

```
./scripts/start_esibot_all.sh        # teleop + IMU + EKF + dashboard (RPi4)

ros2 launch esibot_slam mapping.launch.py start_rviz:=true        # mapping
ros2 run nav2_map_server map_saver_cli -f ~/esibot_ws/maps/test_room
ros2 launch esibot_navigation2 navigation2.launch.py \
    map:=$HOME/esibot_ws/maps/test_room.yaml use_rviz:=true      # navigation
```

Listing 6.1. Full sequence — low-level startup, then mapping and navigation.

The workspace (`~/esibot_ws`) groups the `esibot_*` packages, the `depth_cleanup` library, the third-party dependencies `kinect_ros2`, `esp32_camera_bridge` and `m-explore-ros2`, in addition to the standard ROS 2 Humble packages (`slam_toolbox`, `nav2_bringup`, `robot_localization`, `depthimage_to_laserscan`, `joy`, `teleop_twist_joy`, `cv_bridge`). The `START_EVERYTHING.md` file in `working_build` documents the build sequence, the manual startup commands, the topic checks, and map saving.

CHAPTER 7

Measured Results and System Parameters

7.1 Configuration Values

The operating parameters declared in the architecture dictate the robot's overall behavior. Table 7.1 gathers the key characteristics derived from the system configuration.

Table 7.1. Quantities derived from the configuration parameters.

Quantity	Value	Calculation / source
Effective encoder resolution	24 pulses/rev	`encoder_cpr=12`, dual-edge counting
Control loop / IMU / EKF frequencies	10 / 50 / 30 Hz	`control_hz`, `mpu6050_node`, `ekf_odom_imu.yaml`
Gyroscope bias calibration	100 samples	`gyro_calibration_samples`
Raw `/scan` rate / SLAM-integrated rate	$\approx$30 Hz / $\leq$2 Hz	`scan_time`=0.033 s; interval=0.5 s
Ultrasonic radar rate	$\approx$0.07 Hz	`ultrasonic_radar.yaml`
Max. linear/angular velocity (teleoperation)	0.25 m/s / 2.4 rad/s	`scale_linear.x`, `scale_angular.yaw`
Min./max. linear velocity (Nav2/DWB)	0.2 / 0.3 m/s	`esibot.yaml`
Goal position tolerance (`FollowPath`)	0.05 m	`esibot.yaml`
SLAM/costmap map resolution	0.05 m/cell	consistent `slam_toolbox`/Nav2

Velocity Asymmetry

The maximum angular velocity in manual teleoperation (2.4 rad/s) is set higher than the autonomous speed to ensure strong human responsiveness.

7.2 Validation Plan

Physical validation of the platform relies on the scenarios in Table 7.2, which verify the reliability of each layer.

Table 7.2. Field validation plan.

Scenario	Procedure	Expected criterion
Low-level startup	Start motors, MPU6050, EKF and dashboard	`/odom`, `/imu/data_raw`, `/odometry/filtered` topics stable
Odometric consistency	Travel 2 m in a straight line after PWM calibration	measured and reproducible lateral drift
Kinect pipeline	Compare `/scan` to known wall distances	bounded range error with no persistent gaps
Mapping	Build a Kinect map of a 3×3 m area	continuous walls, plausible loop closure
Navigation	Send three successive Nav2 goals on a saved map	arrival without collision and controlled stop
Exploration	Initialize `m-explore-ros2` with SLAM/Nav2 active	frontier goals issued without TF conflict

Table 7.3. Consolidated validation status in the report.

Item	Current status	Remaining measurement
Motor control	implemented and calibratable	final PWM/RPM curve on the assembled robot
EKF odom+IMU	configured conservatively	drift before/after fusion on a repeated trajectory
Kinect → `/scan`	launched by `mapping.launch.py`	raw/cleaned depth comparison after `depth_cleanup` integration
Nav2	complete configuration	multi-goal trials on the actual EsiBot map
Exploration	dependency added	launch orchestration and frontier testing

CHAPTER 8

Critical Analysis, Limitations and Deviations

This chapter distinguishes the project's deliberate deviations from the limitations that still require experimental validation. The current state remains promising: the software architecture is consistent, the main nodes exist, and the Kinect/Nav2/EKF choices provide a technically sound foundation. The points below mostly concern the fine calibration needed to turn a functional integration into a repeatably demonstrable product.

8.1 Architectural Trade-offs

During development, several adjustments were made relative to the initial concepts:

Table 8.1. Summary of architectural changes.

Aspect	Final implementation
Motor/servo control	Centralized on the Raspberry Pi 4; the ESP32-CAM is dedicated to the video stream.
Mapping sensor	Kinect v1 via `kinect_ros2` and `depthimage_to_laserscan`, with `depth_cleanup` ready for a cleaned stream.
Ultrasonic radar	Kept in the code for testing or as a slow fallback, but not retained as the primary autonomous driving sensor.
Exploration	`m-explore-ros2` referenced by the branch, still to be orchestrated in a complete EsiBot launch.
Web gateway	A dedicated Python HTTP server for telemetry, simpler than the WebSocket gateway originally planned.

These deviations do not degrade the product's coherence. On the contrary, they clarify a more realistic architecture: real-time control stays on the Raspberry Pi, while heavy perception and planning remain offloadable to the operator PC.

8.2 Cross Odometry–SLAM Calibration

The deepest limitation is not a missing module, but the numerical agreement between the topics that describe the same motion. A correct PWM→RPM calibration is not enough if the wheel radius, wheelbase, yaw correction, EKF fusion and `slam_toolbox` thresholds do not converge on the same real-world displacement.

Table 8.2. Parameters to be calibrated jointly.

Chain	Sensitive parameters	Symptom if miscalibrated
Wheels → `/odom`	calibrator radius 0.033 m, `wheel_base`=0.16 m, effective CPR	biased integrated distance or rotation
URDF/TF	0.025 m radius in the model, visual wheelbase 0.14 m	inconsistency between the RViz2 model, costmaps and odometry
IMU → EKF	gyroscope bias, yaw thresholds, `imu0_config`	robot over- or under-corrected when driving straight
`/scan` → map	`minimum_travel_distance`, `minimum_travel_heading`, Kinect range	stretched map, duplicated walls or unstable loop closures
Nav2 → `/cmd_vel`	DWB limits, min. speed 0.2 m/s, accelerations	oscillations or trajectories too aggressive for the motors

The recommended method is manual and based on short ROS 2 bags: record `/cmd_vel`, `/odom`, `/odometry/filtered`, `/scan`, `/tf` and `/map` over a straight line, an in-place rotation, and a reference square; compare the actual distance, the odometric trajectory, and the `map`→`odom` correction; then adjust the radius, wheelbase, and yaw gains before modifying the `slam_toolbox` thresholds. This step requires human analysis, since the same visible defect on the map can stem from a poorly calibrated wheel, a biased IMU yaw, a noisy Kinect scan, or an excessively high update threshold.

8.3 Perception Sensors

The switch from the ultrasonic radar to the Kinect greatly improves autonomous driving, but does not turn the system into a LiDAR. The remaining limitations are known and calibratable:

- **Ultrasonic radar** — the current sweep is slow (≈0.07 Hz), narrow ($60°$) and sequential. It does not provide dense enough observation for a dynamic costmap; it should remain a test tool or a very local fallback.
- **Kinect** — the horizontal field of view of about $57°$, the usable 0.35–4.0 m range and sensitivity to infrared light require active rotation of the robot and indoor use. These limitations are acceptable for the demonstration scenario, since the Kinect still provides a depth scene far more usable than a single ultrasonic beam.
- **`depth_cleanup`** — the library exists and its tests protect the metric format, but the adapter ROS 2 node has not yet been inserted into `mapping.launch.py`. The gain on `/scan` must therefore be measured before the defense.

8.4 Safety and Operations

- **Teleoperation without an enable button** — the Xbox configuration accepts a command as soon as the stick moves. This is convenient during testing, but a demonstration mode should reintroduce a dead-man button or a clear software stop.
- **No battery supervision** — no node publishes `/battery_state`. Long trials must therefore be monitored outside the software to avoid a cutout during navigation.
- **Hardcoded ESP32-CAM WiFi credentials** — acceptable for a local prototype, but should be replaced with a configuration mechanism if the repository becomes public or is reused.

8.5 Performance and Orchestration

Simultaneously running the Kinect, `depthimage_to_laserscan`, `slam_toolbox`, Nav2 and exploration may exceed the comfortable capacity of a Raspberry Pi 4 alone. The three-layer separation therefore remains relevant: the Raspberry Pi drives the hardware and publishes `/odom//imu/data_raw`, while the operator PC can carry the Kinect, SLAM, Nav2, RViz2 and exploration. The point to validate is not only CPU load, but also DDS network latency and TF freshness, especially when exploration sends goals while the map is being built.

8.6 Summary

The product remains optimistic in its expected results: the chosen architecture is better than the initial ultrasonic plan for credible autonomous navigation. The main condition is to treat calibration as a complete experimental campaign, not as an isolated adjustment of the left or right motor. Once this campaign is completed, the remaining limitations mainly concern operational safety and the orchestration of the high-level modules.

CHAPTER 9

Outlook and Conclusion

9.1 Technical Summary

The **EsiBot** project resulted in a functional differential-drive mobile robot aligned with recognized architectures such as the TurtleBot3 Burger. Using a standard ROS 2 software stack (`slam_toolbox`, Nav2, `robot_localization`) to drive a low-cost robot made it possible to deliver:

- a complete, autonomous low-level control chain (encoders, per-wheel PID, tooled PWM→RPM calibration) entirely running on the Raspberry Pi 4;
- a correctly scoped EKF sensor fusion (velocities only for odometry, absolute orientation for the IMU), avoiding double-counting of drift;
- a coherent and correctly parameterized Kinect → `depthimage_to_laserscan` → `slam_toolbox` mapping pipeline (aligned range bounds, loop closure enabled);
- a complete Nav2 configuration (AMCL, dual-layer costmaps, DWB, recovery behaviors, collision monitor);
- functional controller teleoperation and a telemetry web dashboard that are simple to deploy (no external dependency for the HTTP server).

9.2 Prioritized Action Plan

Based on the analysis in Chapter 8, the following corrective actions are proposed in decreasing order of priority:

1. **Reconcile the geometric constants** (wheel radius, wheelbase, GPIO assignments) between the URDF, `motor_bridge_node` and `motor_calibrator`, ideally via a single shared parameter file.
2. **Conduct the field measurement campaign** described in Chapter 7 (Tables 7.2 and 7.3), in particular the actual mapping of a 3 m×3 m area with the Kinect, followed by validation of autonomous navigation on that map — the most urgent priority before the defense, as the jury's demonstration scenario depends directly on it.
3. **Implement battery supervision** (`/battery_state`) to secure trials and demonstrations.
4. **Add a minimal vision node** (line or color detection via OpenCV) consuming `/esp32/camera/image_raw`, even if only for demonstration purposes, to close the gap with Deliverable 3 of the specification.
5. **Extend the web dashboard** to include, at minimum, a display of the SLAM map (canvas rendering of the `OccupancyGrid`) and a low-frequency preview of the camera stream, bringing Deliverable 5 closer to its full specification.
6. **Reintroduce an enable button** for controller teleoperation, or otherwise explicitly document this design choice in the README (Deliverable 8).

9.3 Future Outlook

Beyond the immediate corrections, several avenues for evolution could enrich **EsiBot** in the remainder of the project or for future cohorts: the joint use of the Kinect's color stream (currently only depth is used) for visual detection complementary to the ultrasonic radar kept in reserve; the reintroduction of three-sensor fusion (Kinect + ultrasonic radar + odometry) for increased robustness in the event of a temporary failure of one source; the addition of a complete power supervision system with estimation of remaining autonomy; and, more broadly, consolidating the documentation (README, formal electronic schematic) to make the project more reliably reproducible by other teams, in keeping with the open-source spirit claimed from the initial specification onward.

9.4 Concluding Remarks

This report has documented **EsiBot** as it was actually designed and coded, rather than as it had been initially envisioned on paper — a deliberate methodological choice that, the team hopes, constitutes a more useful foundation for the rest of the project than mere reproduction of the original specification. The identified deviations are not failures but traceable and justifiable engineering decisions; the action plan above sets out the path toward complete experimental validation of the system before the final defense.

APPENDIX A

Technical Appendices

A.1 Exhaustive Table of ROS 2 Topics

Table A.1. Exhaustive list of ROS 2 topics identified in the source code.

Topic	Type	Published by	Subscribed by
/cmd_vel	geometry_msgs/Twist	teleop_twist_joy_node, nav2	motor_bridge_node
/odom	nav_msgs/Odometry	motor_bridge_node	ekf_filter_node
/odometry/filtered	nav_msgs/Odometry	ekf_filter_node	nav2, dashboard
/encoder_ticks\protect\penalty\z@(_total)\protect\penalty\z@/left,right	Int32/Int64	motor_bridge_node	(diagnostics)
/imu/data_raw	sensor_msgs/Imu	mpu6050_node	motor_bridge_node, ekf, dashboard
/imu/temperature	sensor_msgs/Temperature	mpu6050_node	dashboard
/scan	sensor_msgs/LaserScan	depthimage_to_laserscan (Kinect) *or* ultrasonic_scan_node (deprecated)	slam_toolbox, nav2
/kinect/depth/image_raw, /camera_info	Image / CameraInfo	kinect_ros2_node	depthimage_to_laserscan
/map	nav_msgs/OccupancyGrid	slam_toolbox	nav2, RViz2
/tf, /tf_static	tf2_msgs/TFMessage	robot_state_publisher, ekf, slam_toolbox	nav2, RViz2
/joy	sensor_msgs/Joy	joy_node	teleop_twist_joy_node, dashboard
/esp32/camera/image_raw	sensor_msgs/Image	esp32_camera_bridge	(optional consumer)
/goal_pose	geometry_msgs/PoseStamped	RViz2	nav2 (bt_navigator)
/global,local_costmap\protect\penalty\z@/costmap	nav2_msgs/Costmap	nav2 (costmap servers)	RViz2, planner/controller
/plan	nav_msgs/Path	planner_server	controller_server, RViz2

A.2 Dashboard HTTP API and Repository ROS 2 Packages

The imu_web_dashboard server (port 8088 by default) exposes three endpoints: GET / (or /index.html), the dashboard HTML/JS page; GET /api/imu, the consolidated JSON state (IMU, odometry, controller, /cmd_vel); and POST /api/zero, resetting the displayed roll/pitch offsets.

Table A.2. ROS 2 packages of the `Esi_Bot` workspace.

Package	Role
`esibot_bringup`	orchestration launches (teleoperation, IMU, EKF, dashboard, radar)
`esibot_hardware`	motor/encoder control (`motor_bridge_node`), `pwm_mapper`, `motor_calibrator`
`esibot_sensors`	`mpu6050_node`, `ultrasonic_scan_node` (deprecated)
`esibot_description`	URDF/xacro model, `robot_state_publisher` launch
`esibot_slam`	launch of the Kinect → `depthimage_to_laserscan` → `slam_toolbox` pipeline
`esibot_navigation2`	Nav2 launch, `esibot.yaml` parameters, default map
`esibot_dashboard`	`imu_web_dashboard` (telemetry HTTP server)
`depth_cleanup`	C++/OpenCV library for metric cleanup of `16UC1` Kinect images
`kinect_ros2` (external)	Kinect (v1) driver, based on `libfreenect`
`m-explore-ros2` (external)	frontier-based autonomous exploration on top of Nav2 and `/map`
`esp32_camera_bridge` (external)	MJPEG HTTP → ROS 2 `Image` bridge

A.3 Sources

All of the technical content in this report was established from the following sources, provided by the team: the `Esi_Bot` source code repository (`working_build` branch, `esibot_*` packages), the `kinect_ros2` driver, the firmware and ROS 2 bridge of the ESP32-CAM (`esp32-camera`), as well as the official **EsiBot** project specification sheet (requirements document, TurtleBot3 comparison, list of deliverables) and the two architecture diagrams provided by the team (Figures 2.1 and 2.2). The Kinect-versus-ultrasonic-sensor discussion also draws on the article by Kamarudin, Mamduh, Shakaff and Zakaria, *Performance Analysis of the Microsoft Kinect Sensor for 2D Simultaneous Localization and Mapping (SLAM) Techniques*, *Sensors* 14(12), 23365–23387, 2014, DOI `10.3390/s141223365`.